



SÃO
PAULO
TECH
SCHOOL

Qualidade de Software

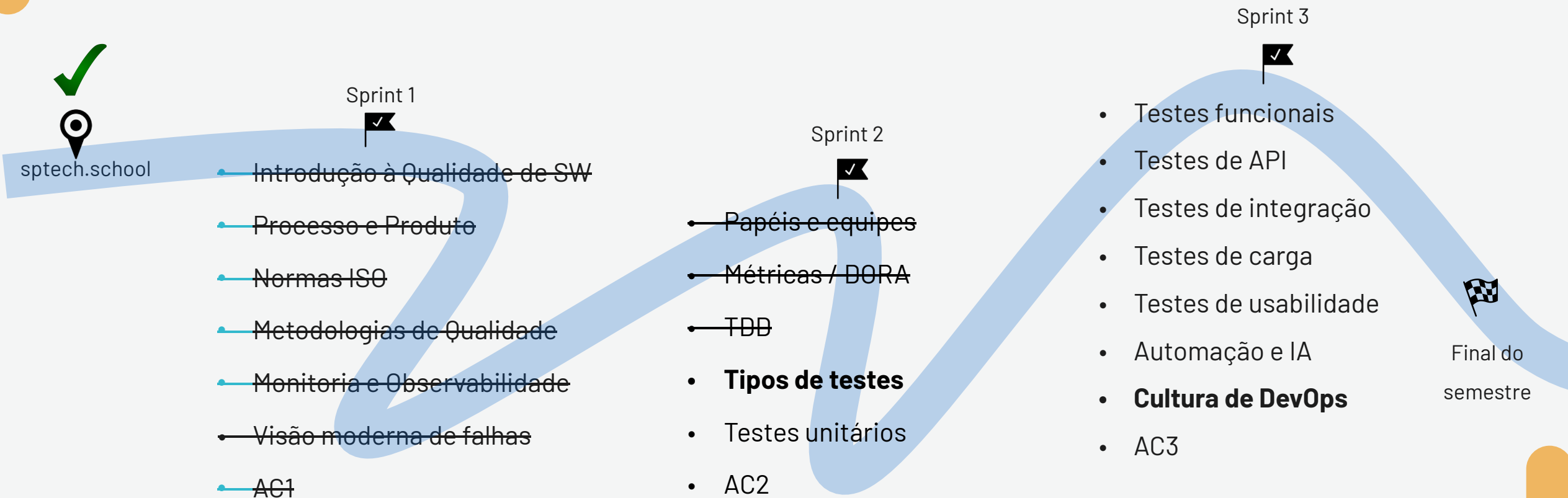
Cultura de DevOps

Tipos de testes

7/abr

Prates

Nosso caminho - Ementa



Tópicos da Aula

Pilares da cultura DevOps

Automação do ciclo de desenvolvimento

Trunk Based Development e Feature Flags

Deploys frequentes e pequenos

Tipos de testes

Atividade



Pilares da cultura Dev Ops



Por que DevOps surgiu?

Em muitos contextos, desenvolvimento, testes e operações trabalhavam separados

Isso gerava:

- entregas lentas
- muita burocracia / retrabalho
- falhas descobertas tarde
- deploys grandes e arriscados

Resultado: baixa previsibilidade e dificuldade para manter qualidade

Por que DevOps surgiu?

Working in Silos





O que DevOps tem a ver com Qualidade?

Qualidade não é só validar o software; é construir um processo capaz de entregar com confiança.

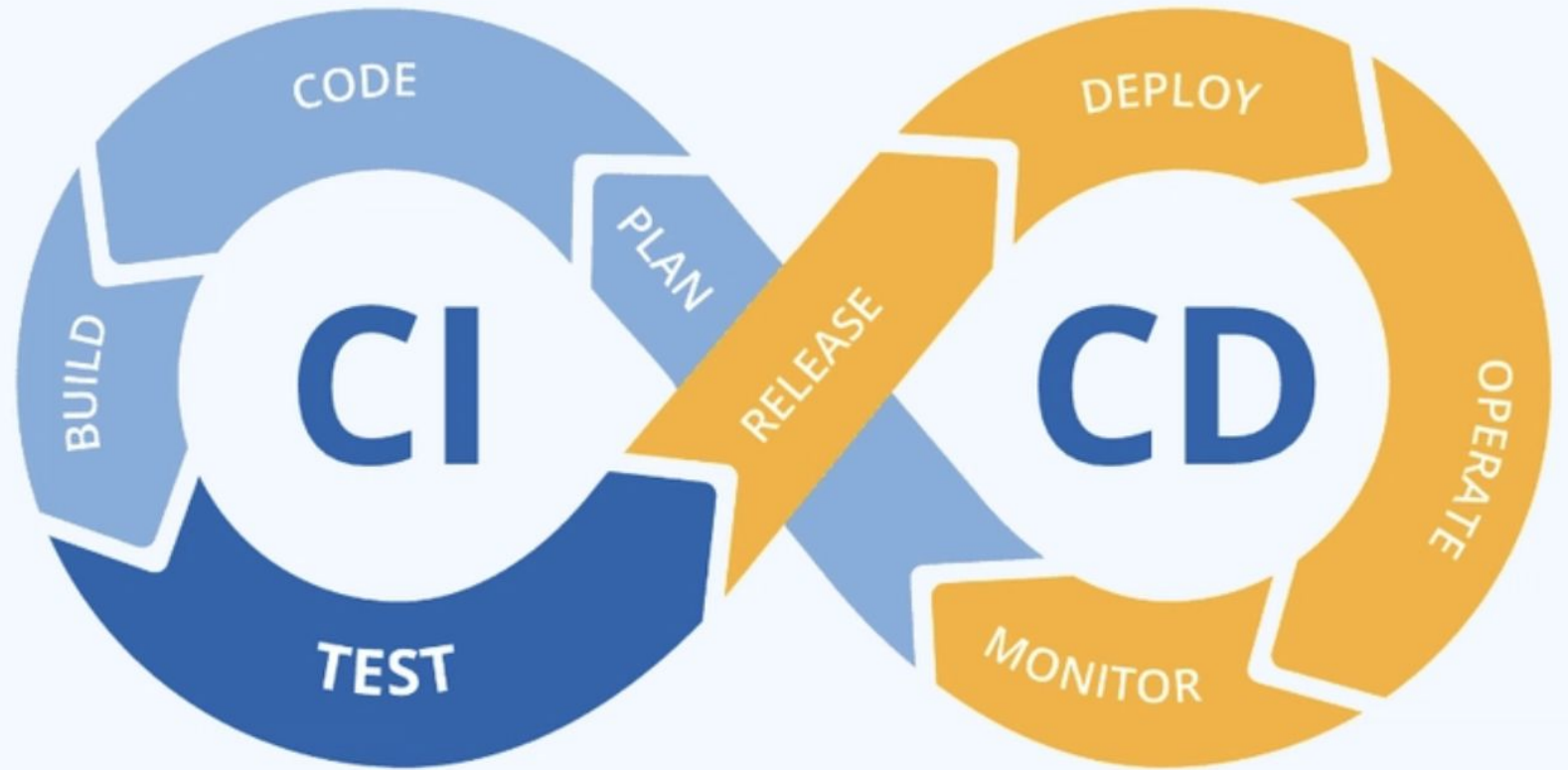
Qualidade não depende apenas de testar no fim.

Qualidade melhora quando o processo inteiro é mais confiável.

DevOps ajuda porque:

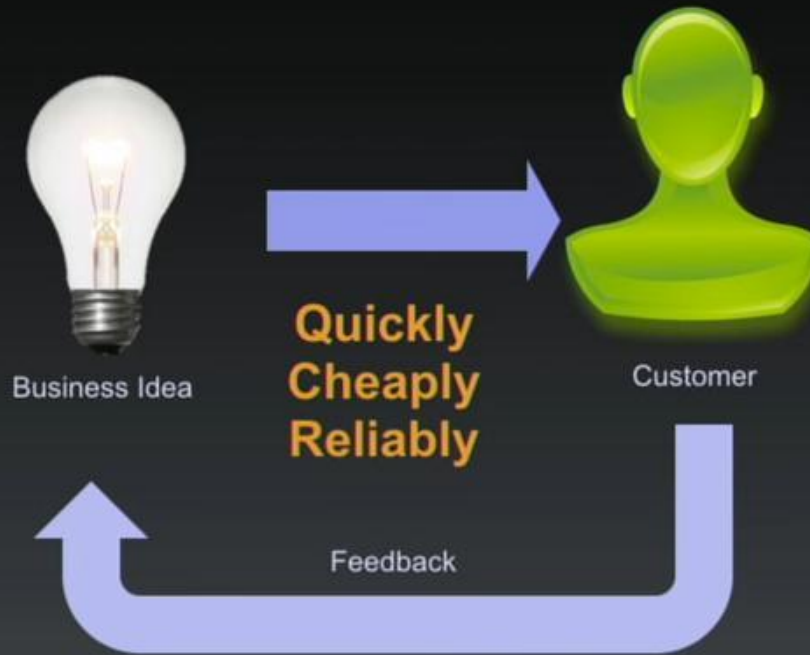
- reduz tempo entre mudança e feedback
- estimula automação
- reduz erro manual
- facilita rollback
- aproxima quem cria de quem opera

Ciclo completo e integrado



O que nós queremos?

What Do We Really Want?



O que é a cultura DevOps?



Cultura DevOps

A cultura DevOps não se trata apenas do uso de ferramentas, nem de automação de pipelines.

É uma cultura que integra:

- desenvolvimento
- testes
- operações
- colaboração
- automação
- melhoria contínua



Pilares da cultura de DevOps

Os principais pilares da cultura DevOps são:

- Colaboração entre desenvolvimento, testes e operações
- Automação do ciclo de desenvolvimento
- Responsabilidade compartilhada
- Feedback rápido
- Melhoria contínua
- Shift left na qualidade
- “You build it, you run it”



Comparativo dos modelos

Aspecto	Tradicional	DevOps
Times	Separados	Integrados
Testes	No final	Durante todo o ciclo
Deploy	Raro e grande	Frequente e pequeno
Automação	Baixa	Alta
Feedback	Lento	Rápido
Risco	Maior	Menor
Correção de falhas	Mais lenta	Mais rápida
Qualidade	Etapa isolada	Responsabilidade contínua

Automação do ciclo de desenvolvimento



Automação do ciclo de desenvolvimento

Por que automatizar?

- Velocidade
- Redução de erros humanos
- Reprodutibilidade
- Tornar o processo previsível
- Aumentar confiança nas entregas

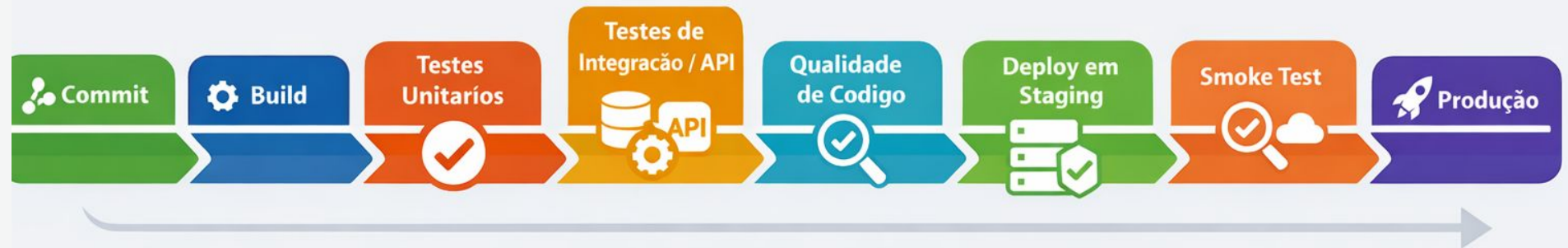


Automação do ciclo de desenvolvimento

O que automatizar?

- Build da aplicação
- Execução de testes
- Análise estática de código
- Provisionamento de ambiente
- Deploy
- Rollback
- Observabilidade inicial
- Rotinas operacionais repetitivas

Exemplo de Pipeline de CI/CD



Cada etapa impede que uma mudança ruim avance.



Automação do ciclo de desenvolvimento

Ferramentas comuns utilizadas

- CI/CD: Jenkins, GitHub Actions, GitLab CI/CD, Azure DevOps
- Qualidade de código: SonarQube
- Feature flags: LaunchDarkly, Split
- Containers/ambientes: Docker
- Observabilidade: Grafana, Prometheus, Datadog

The background is a dark, almost black, space filled with intricate, glowing patterns. These patterns consist of numerous thin, white, curved lines that flow and swirl across the frame, creating a sense of dynamic movement. Interspersed among these lines are many small, bright white dots and star-like particles, some of which appear to be trailing or leaving a wake behind them. The overall effect is reminiscent of a cosmic nebula, a data visualization, or a complex network of energy. The lighting is soft yet highlights the edges of the flowing lines, giving them a three-dimensional appearance.

Trunk Based Development

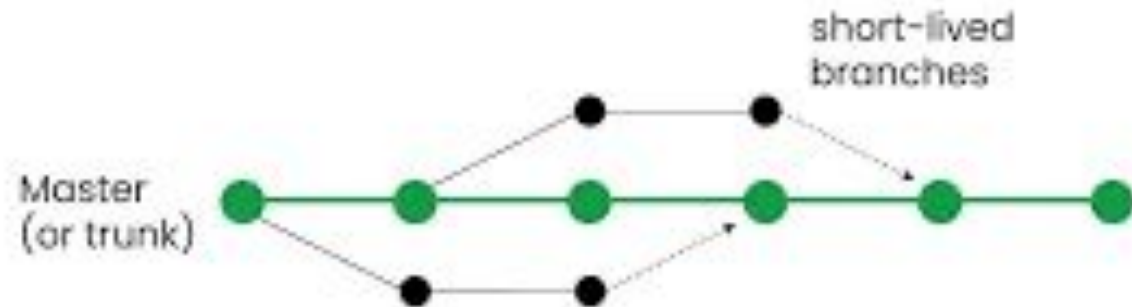


Trunk based development

O que é Trunk-Based Development?

- Modelo em que o time integra no tronco principal com frequência
- Branches curtas e temporárias
- Merges frequentes
- Menor tempo de divergência
- Melhor encaixe com integração contínua

Trunk based development



Trunk Based Development





Trunk based development

Benefícios do Trunk-Based Development

- Menor risco de conflitos de código
- Redução de divergências entre branches
- Acelera a entrega de funcionalidades
- Permite que o software seja entregue a qualquer momento

Feature Flags

The background is a dark, textured surface with intricate, flowing, and wavy lines in shades of gray and white. These lines create a sense of movement and depth. Scattered throughout the background are numerous small, bright white dots and larger, out-of-focus circular bokeh lights, giving it a cosmic or digital feel.



Feature Flags

O que são Feature Flags?

Mecanismo para ativar ou desativar funcionalidades sem novo deploy. Separa “código implantado” de “funcionalidade liberada”.

Permite liberar por:

- grupo de usuários
- região
- porcentagem da base



Feature Flags

Benefícios das Feature Flags

- Controle sobre o lançamento de funcionalidades
- Redução de riscos em deploys
- A/B Testing
- Rollback funcional rápido



Feature Flags

Boas Práticas com Feature Flags

- Manter o código limpo, remover flags após o lançamento
- Gerenciar adequadamente as flags
- Monitorar o impacto das novas funcionalidades
- Controle de cohorts, grupos de usuários que acessam cada feature (eventualmente impacta no suporte)

The background is a dark, textured surface with intricate, flowing, and swirling patterns of light gray lines. These lines create a sense of movement and depth, resembling a topographical map or a fluid simulation. Scattered throughout the background are numerous small, bright white dots and larger, soft, out-of-focus circular bokeh lights, adding a celestial or digital aesthetic to the overall composition.

**Deploys pequenos e
frequentes**



Deploys pequenos e frequentes

Por que Deploys Frequentes?

- mudanças menores são mais fáceis de testar
- quando algo quebra, fica mais fácil descobrir a causa
- o rollback é menos traumático
- o time recebe feedback mais rápido
- há menos acúmulo de risco



Deploys pequenos e frequentes

Como Implementar Deploys Frequentes

- pipeline automatizado
- testes rápidos e confiáveis
- trunk-based development
- feature flags
- monitoramento constante
- rollback seguro

Tipos de Testes

The background is a dark, monochromatic abstract composition. It features a series of thin, white, wavy lines that flow across the frame, creating a sense of movement and depth. Interspersed among these lines are numerous small, out-of-focus light points, resembling bokeh or distant stars, which add to the ethereal and high-tech aesthetic of the image.



Tipos de testes

Quais tipos de testes usamos em software?

- unitários
- integração
- API
- funcionais / aceitação / E2E
- carga
- usabilidade
- mobile

Pirâmide de Testes





Testes unitários

- Validam uma unidade pequena de código
- São rápidos e baratos
- Devem isolar dependências sempre que possível
- Bons para regras de negócio

Exemplos:

- calcular desconto
- validar CPF
- impedir quantidade negativa no carrinho
- calcular frete mínimo



Testes de integração

- Validam a interação entre componentes
- Respondem à pergunta: “essas partes funcionam juntas?”
- Normalmente envolvem banco, fila, serviço externo ou módulo interno

Exemplos

- salvar pedido e atualizar estoque
- cadastrar usuário e persistir no banco
- consumir serviço de pagamento e gravar status



Testes de API

- Validam contratos, status code, payload e comportamento
- São mais rápidos que E2E e mais realistas que muitos testes unitários
- Muito úteis em arquitetura distribuída

Exemplos

- POST /login retorna 200 para credenciais válidas
- POST /pedido retorna 400 com payload inválido
- GET /produtos/{id} retorna o produto correto



Testes funcionais e de aceitação

- Validam o sistema do ponto de vista do usuário
- Cobrem fluxos completos
- Têm alto valor de negócio, mas são mais lentos e frágeis (precisam de muito setup e manutenção)
- Devem ser poucos e estratégicos

Exemplos

- usuário faz login e finaliza compra
- usuário recupera senha com sucesso
- usuário cadastra endereço e conclui checkout



Testes de carga, usabilidade e mobile

Carga

- mede comportamento do sistema sob alto volume (ex.: 2.000 usuários simultâneos no checkout)

Usabilidade

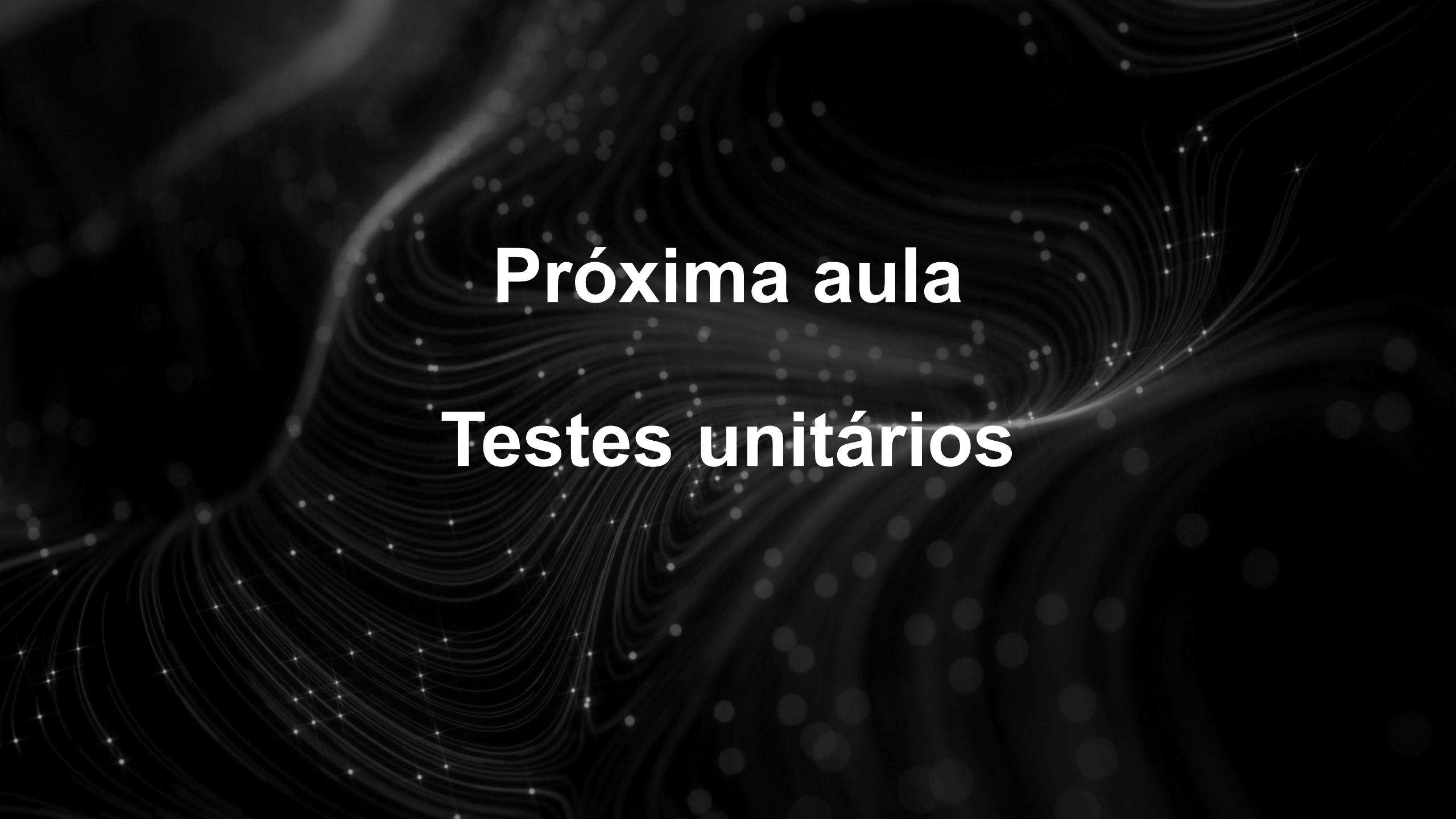
- avalia facilidade de uso e experiência (ex.: usuário encontra ou não o botão de finalizar)

Mobile

- avalia comportamento em telas, dispositivos e contextos móveis (ex.: app em Android/iOS, perda de conexão, rotação de tela)

Atividade dia 8



The background is a dark, textured surface with intricate, flowing, and swirling patterns of light gray lines. These lines create a sense of movement and depth, resembling a nebula or a complex data visualization. Scattered throughout the background are numerous small, out-of-focus light spots, known as bokeh, which vary in size and brightness, adding to the ethereal and futuristic feel of the image.

Próxima aula

Testes unitários



Agradeço a sua atenção!

Qualidade de Software

Professor Prates

Aula 8

SÃO
PAULO
TECH
SCHOOL